Winding Tree

Building trust on a trustless blockchain

Jiří Chadima · Follow

Published in Winding Tree · 6 min read · Apr 24, 2019

In the previous post, I had talked about why Winding Tree needs a way to give the ecosystem participants information on which they can decide who to trust. I also proposed a solution with asymmetric cryptography and a smart contract called OTA Index. Today, I'm going to elaborate on that a little more and show you how we actually plan to do it. Don't worry, there are pictures.

Note: *This can still change if we hit some serious obstacle during the implementation, but hopefully, we won't.*

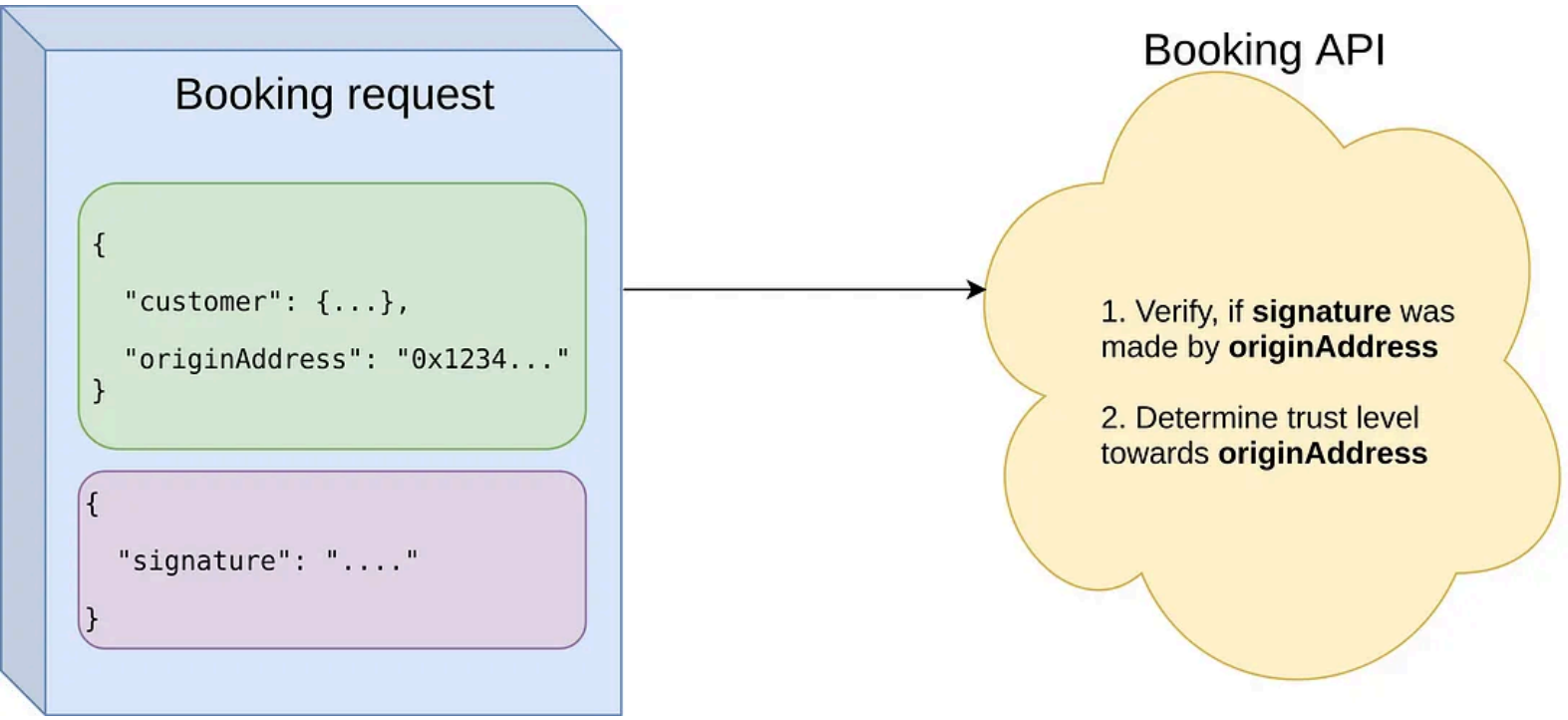
The proposal from the first article is basically trying to answer only one question: ***Do I trust the data that I'm getting on Winding Tree?*** This data, of course, being (but not limited to) an incoming booking request or the data published about a hotel in the Winding Tree index.

1. Who's talking to you?

As said in the first article, this is a perfect use case for asymmetric cryptography. And since we're working on top of Ethereum blockchain, we can easily benefit from every ETH address being essentially a hashed public key which can be used to verify messages signed by the appropriate private key.

Booking requests

Every booking request will have to contain an ETH address and will have to be signed by the appropriate private key. That way, Bob who's running it, can positively tell who is sending the request (it's not an impostor) and gets an ETH address towards which he has to establish trust.



Booking requests need to contain signature so that Booking API can positively tell who is calling it

Hotel data

If you remember, Alice was running an OTA (Online Travel Agency). And now she wants to know if she can trust a hotel owned and published by Bob. In the current data model, Alice could try to establish her trust level towards the hotel manager's address (an owner of the hotel record).

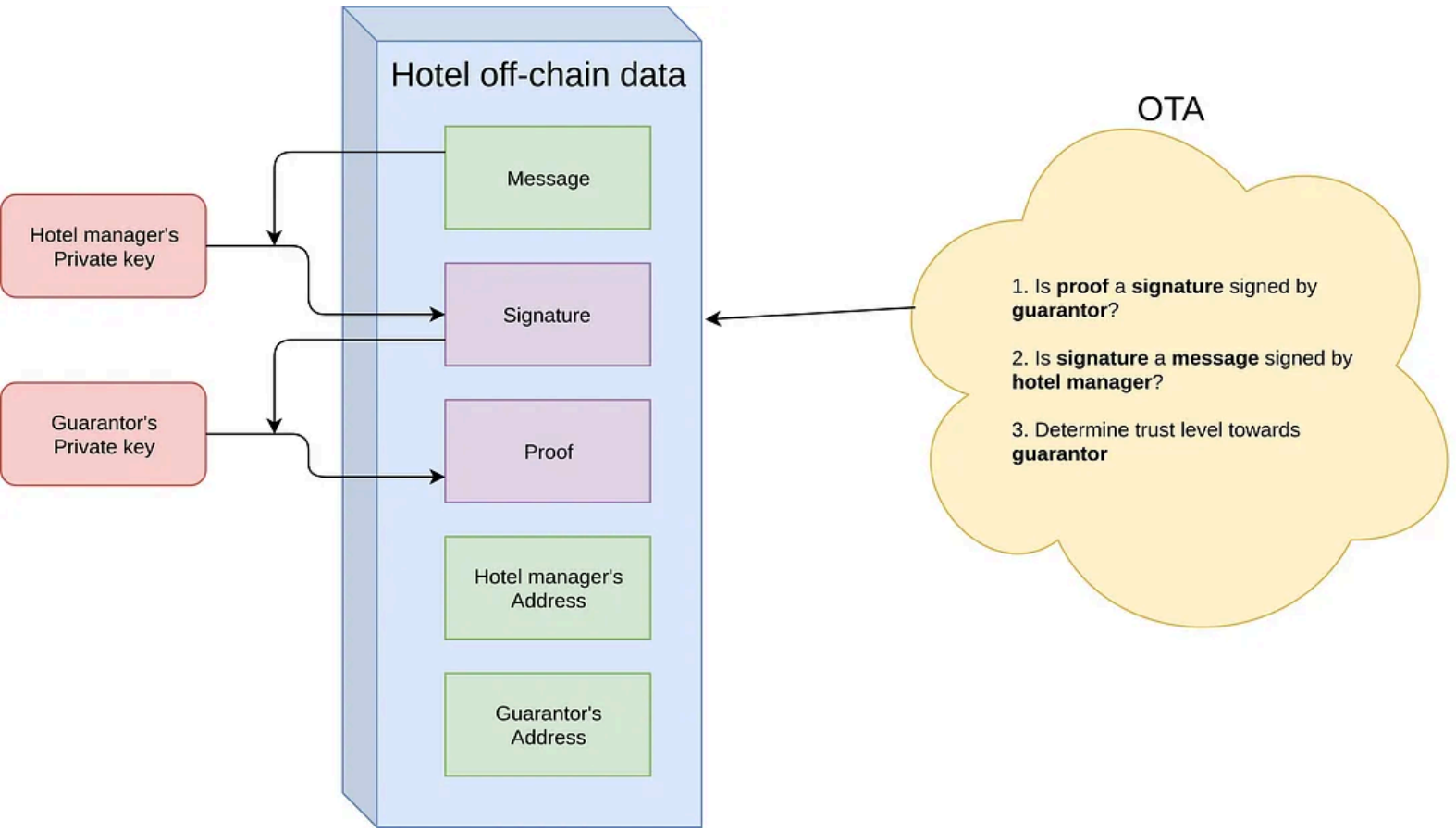
But this approach might be very limiting. What if Bob owns multiple hotels under the same brand? That would mean that he would have to create a level of trust for every single hotel he owns. And that might not be feasible.

So we propose that Bob can delegate the trust to a third ETH address (not the hotel's address, not the hotel manager's address). Let's call this third address Hillary. (It's similar to franchising a branded fast food. Someone else is providing a trusted brand while you are running the restaurant.)

In order to prove that Hillary is providing a guarantee for Bob's hotel, Bob has to disclose some information within the off-chain hotel data:

```
{
  "guarantor": "0xD037aB9025d43f60a31b32A82E10936f07484246",
  "message: "Pre-agreed message signed first by hotel owner and then
by hotel guarantor, it might be a hotel name, it might contain other
information",
  "signature":
"0xb91467e570a6466aa9e9876cbcd013baba02900b8979d43fe208a4a4f339f5fd60
07e74cd82e037b800186422fc2da167c747ef045e5d18a5f5d4300f8e1a0291c",
  "proof":
"0xd91467e570a6466aa9e9876cbcd013baba02900b8979d43fe208a4a4f339f5fd60
07e74cd82e037b800186422fc2da167c747ef045e5d18a5f5d4300f8e1a0291c"
}
```

Alice now needs to first make sure that *proof* is actually a *signature* signed by *guarantor*, and then verify that *signature* is actually a *message* signed by the hotel *manager*. This two step signature ensures that Bob actually talked to Hillary and she agreed to provide a guarantee for him.



An intricate signature scheme

Why this complicated? We need to prove that the guarantor actually talked to the hotel. At the same time, we need to prevent other hotels to provide proofs done by a guarantor for another hotel. We also assume that the signing ritual between a hotel manager and guarantor does not happen very often — even so, the signatures should probably expire after some time, ideally, the expiration would be a part of the signed data.

Of course, the most trivial way of doing this is if Hillary and Bob are actually the same person with the same private key and the data is simply signed twice.

There might be better cryptographic tools for this particular task, but since we want to benefit from the way Ethereum is using signatures, this double envelope scheme might work just fine.

2. Do you trust them?

So we have an ETH address — the *booking originator* or the *hotel guarantor*. The next question Bob or Alice have to answer is: Do I want to trust it?

This is a pretty complex question in real life — we base our trust on personal experience, knowledge shared by people we know, maybe reviews by generally trusted authorities, or even reviews on the internet. But how do we transfer this to a trustless world of software and blockchain?

Surprisingly, in a very similar way. Let's start from Bob who has the ETH address that he should trust. Or maybe trust just a little. That's up to Bob. Now, let's imagine that Bob is running a Booking API. Inside that API, there's already code that spits out the ETH address.

In the next step, the Booking API tries to collect clues that are important to Bob because he told the Booking API when he was configuring it. It might be:

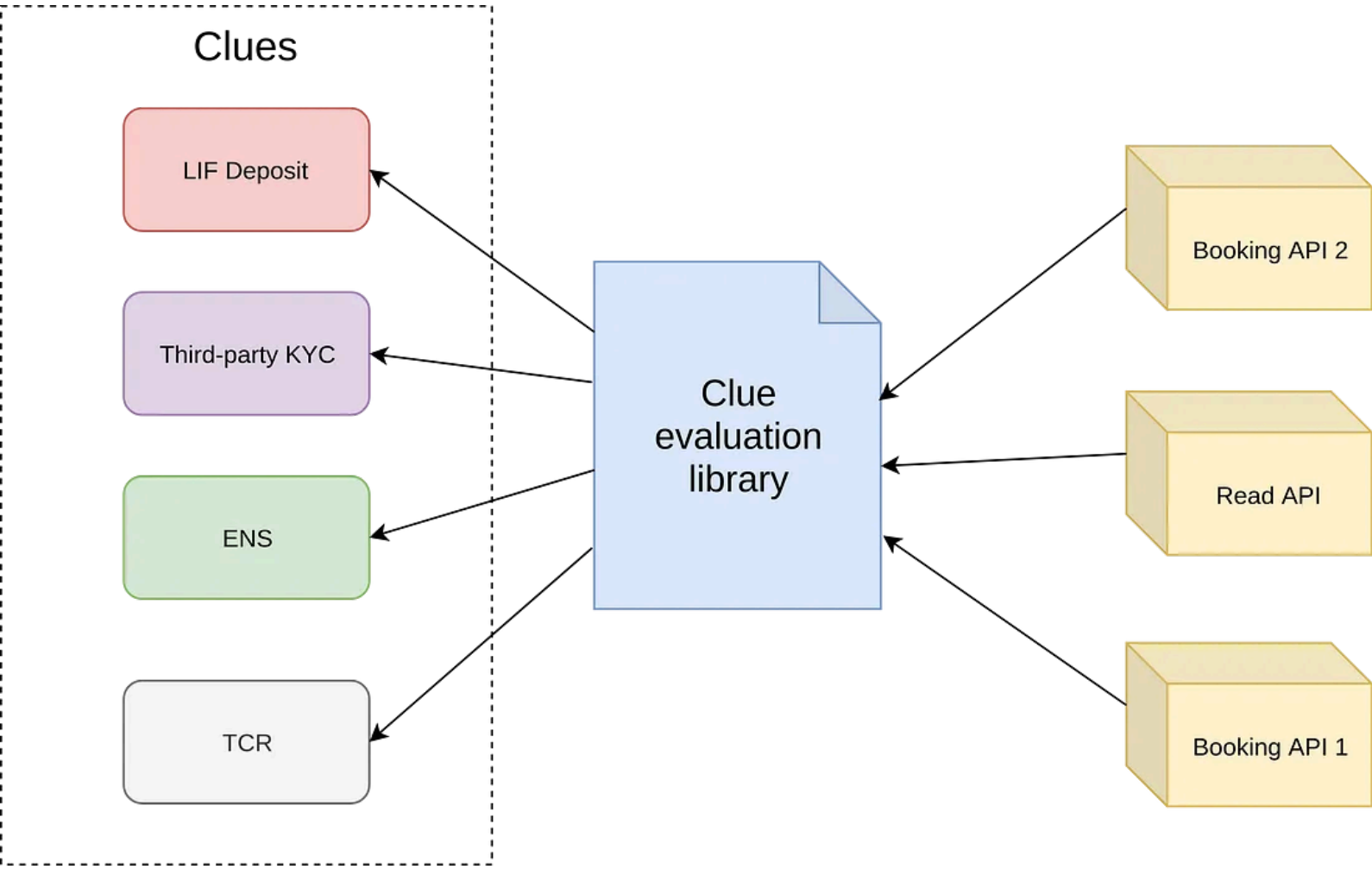
- Has the address been verified by a trusted third party? For example in a Know your customer (KYC) process.
- Is there any LIF associated with that address? Is there enough LIF?
- Has the address been verified in ENS?
- And many more. Perhaps based on Ethereum blockchain, perhaps based on something totally different accessible automatically by software.

After collecting everything that Bob considers important, the Booking API has to make a decision. This by itself is pretty complex, so we're not going to go into details here — it's up to Bob, after all. In the end, it can be a binary decision (trust or no trust) or it can land on a scale somewhere and the requesting party has to for example pay beforehand or something.

Where will you put all of this?

Our intention is to allow anybody to add their own clues into the system. So, similarly to our off-chain data adapters, we will define an interface for a clue

that anybody will be able to use within the Winding Tree tooling. With that, we are not limiting ourselves to blockchain-based clues only.



Stitching everything together

So far, we plan to use this in the Booking API to categorize incoming booking requests and in the Read API to decide which hotels we consider legitimate and which are just test or even fraud attempts.

Our initial release of all of this should also contain implementations of an example on-chain registry and Lif staking or deposit that would fulfill our interface and will be used in all of the tooling operated by us.

Other actors

Online Travel Agencies (OTA)

Let's become Alice for a moment once again. She is running an OTA and wants to send Bookings to many hotels. How or where does Alice present her

clues for all the Bobs out there to consume?

There needs to be some shared knowledge or a curated list of widely used or accepted clues with documentation on how to work with them from Alice's point of view. Winding Tree can manage such list — and if it's going to be a text file on our web, a smart contract or something else, who knows... This part is still very open-ended.

Also, where did the OTA index go? Well, we kind of don't need it right now. It would make much more sense if we wanted to share some data about the OTA with the hotel. We can then easily introduce the OTA index that would work in a similar fashion as the Hotel index, where each OTA would have some off-chain data.

Property management System (PMS)

This is essentially Bob and his Booking API. We intend to also use whitelisting and blacklisting of ETH addresses in the Booking API, there's also room for some machine learning algorithms. The combinations are virtually endless on this end...

. . .

As you can see, this is a complex topic with many actors and many intricate details. We haven't even started to talk about possible options in Lif staking or deposits.

However, we are now very confident that we came up with an architecture that is flexible, easily extendable and more importantly, we can deliver it in a reasonable amount of time. There are still details that we need to experiment with before we find the most appropriate solution, but those really are just details.

Do you have something to add or would you like to ask about more details? Join our Telegram, any technical discussion should go to our Google Group.

If you want to contribute, a good place to start is our GitHub.

Blockchain | Cryptography | Travel Industry | Travel Technology | Wtdeveloper

Published in Winding Tree

1.7K followers · Last published May 5, 2024

Following

Decentralized Travel Distribution

Written by Jiří Chadima

30 followers · 2 following

Follow

Talk is cheap, show me the code. https://www.fragaria.cz/

No responses yet

Maksim Izmaylov

What are your thoughts?

More from Jiří Chadima and Winding Tree

In Winding Tree by Jiří Chadima

2018 Hack Travel — Tech recap

As you can read in our press release, we had a blast during our first official Hackathon held...

Dec 10, 2018 25

In Winding Tree by Marina Berezovska

Announcing Winding Tree #HackTravel Hackathon in Lisbon

We're taking our second hackathon on a tour —join us on July 3–5 in Lisbon, Portugal for ...

Apr 17, 2019 189 1

In Winding Tree by Marina Berezovska

Liquidity Network joins #HackTravel hackathon

We're stoked to welcome Liquidity Network, off-chain payment hub, to join us at the...

Oct 19, 2018 156 1

In Winding Tree by Jiří Chadima

Development process in Winding Tree

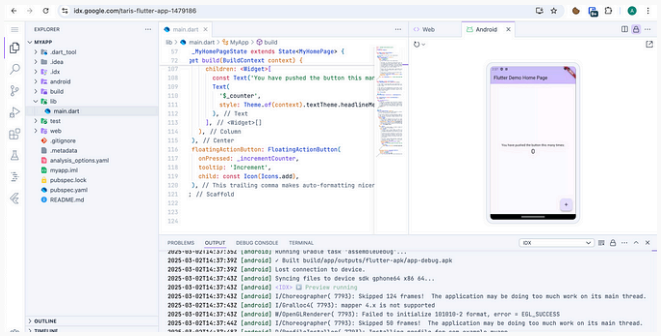
In every project, there is always a communication gap between developers an...

Oct 18, 2018 148

See all from Jiří Chadima

See all from Winding Tree

Recommended from Medium



In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

Mar 11 5.1K 293

In Learn AI for Profit by Nipuna Maduranga

You Can Make Money With AI Without Quitting Your Job

I'm doing it, 2 hours a day

Mar 25 8.6K 398

In Write A Catalyst by Adarsh Gupta

Google just Punished GeekforGeeks

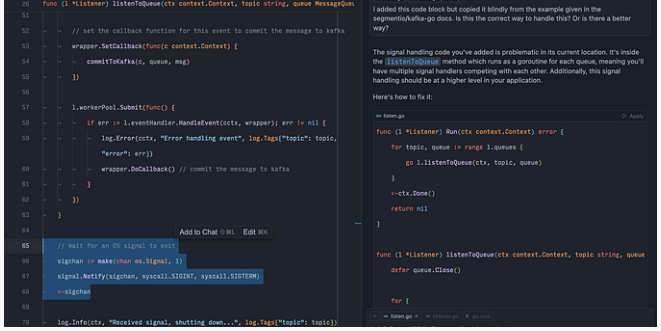
And that's for a reason.

Apr 10 2.9K 126

Daniel B. Gallagher

I worked for Pope Francis. Here is what he was really like.

Apr 22 15.8K 316



In Level Up Coding by Jacob Bennett

The 5 paid subscriptions I actually use in 2025 as a Staff Software...

Tools I use that are cheaper than Netflix

Jan 7 12.9K 318

Kawthar Ahmed

Your Shame Is Not Mine To Carry

There are things I've been called;

6d ago 4.1K 67

See more recommendations